MERN STACK TECHNICAL INTERVIEW STUDY GUIDE

Student Management System — Frontend Guide

Complete File & Folder Explanations, React 18 Hooks, Axios Integration & Q&A

1. High-Level Frontend Architecture

The frontend is a Single Page Application (SPA) built with React 18 and bundled using Vite. Unlike traditional multi-page websites, the browser never reloads the whole page. React dynamically manipulates the DOM in memory via the Virtual DOM based on reactive state changes and backend API responses.

```
Architecture & Data Flow:
[User Action]  % % >  Clicks Add/Edit/Delete or Types in Search
      %
[React State]  % % >  useState updates 'formData', 'students', or 'search'
      %
[Service API]  % % >  services/studentService.js (Axios Client)
      %
[Express API]  % % >  REST Endpoints at http://localhost:5000/api/students
      %
[MongoDB Atlas]% % >  Performs live CRUD queries & returns JSON response
      %
[UI Re-render] % % >  Virtual DOM diffs changes and updates table instantly
```

2. Directory & File-by-File Breakdown

Ø=ÜÁ index.html (Application Root Document)

The sole HTML file loaded by the browser. Contains where React mounts the component tree, and loads src/main.jsx as an ES module.

Ø=Ü¡ Why is index.html mostly empty in a React project?

Answer: React is a Single Page Application framework. Instead of requesting separate HTML files from a server for each view, React injects dynamic JavaScript components into this single root container.

Ø=ÜÁ package.json (Dependencies & Scripts)

Defines project metadata and dependencies. Key production dependencies include "react", "react-dom", and "axios". Development dependencies include "vite" and "@vitejs/plugin-react".

Ø=Ü¡ What is the difference between dependencies and devDependencies?

Answer: dependencies are bundled into the production output because the browser executes them (React, Axios). devDependencies are tools needed only on the developer machine to build, compile, and run the project (Vite).

Ø=ÜÁ vite.config.js (Build Tool & Dev Server Configuration)

Configures Vite plugins, local development port (5173), and the reverse proxy to forward /api requests to Express (:5000) during local development.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: { '/api': 'http://localhost:5000' }
  }
});
```

Ø=Ü¡ Why use Vite instead of Create React App (Webpack)?
Answer: Vite uses native browser ES Modules and the fast Esbuild pre-bundler. It offers instantaneous server startup and near-instant Hot Module Replacement (HMR) without re-bundling the whole project.

Ø=ÜÁ src/main.jsx (JavaScript Entry Point)

Initializes React 18 concurrent root via ReactDOM.createRoot() and wraps the <App /> component inside <React.StrictMode>.

Ø=Ü¡ What does <React.StrictMode> do?
Answer: StrictMode is a developer tool that activates extra checks. In development, it mounts components twice to help identify uncleaned side-effects, memory leaks, and deprecated lifecycle usage.

Ø=ÜÁ src/services/studentService.js (API Service Layer)

Centralizes all Axios HTTP requests into discrete functions (getStudents, addStudent, updateStudent, deleteStudent).

```
import axios from 'axios';
const API_URL = 'http://localhost:5000/api/students';

export const getStudents = (search = '') => axios.get(search ? `${API_URL}?search=${search}` :
API_URL);
export const addStudent = (data) => axios.post(API_URL, data);
export const updateStudent = (id, data) => axios.put(`${API_URL}/${id}`, data);
export const deleteStudent = (id) => axios.delete(`${API_URL}/${id}`);
```

Ø=Ü¡ Why separate API calls into a service file?
Answer: Adheres to the Single Responsibility Principle. Components stay focused purely on rendering UI, while network logic and endpoint URLs are isolated in one easily maintainable place.

Ø=ÜÁ src/App.jsx (Main Component & State Orchestrator)

Holds the central state of the application: student roster (students), search query (search), modal visibility (showModal), edit target (editId), form inputs (formData), and feedback alerts (message).

Uses useEffect to trigger fetchStudents(search) on mount and on search changes. Handles both Create and Update operations dynamically in handleSubmit.

Ø=ÜÁ src/index.css (Enterprise Blue-and-White Design System)

Provides clean styling with CSS Flexbox layouts, modal backdrop overlays (position: fixed with rgba background), responsive table formatting, badge pills, and color-coded alert notifications.

3. Core React Concepts Deep-Dive

• Controlled Components: Input elements have their value bound to React state (value={formData.fullName}) and update via onChange={handleChange}. React is the single source of truth.

• Computed Property Names: In handleChange, [e.target.name]: e.target.value dynamically updates the exact property in formData matching the input name, using the spread operator (...formData) to preserve other fields.

• useEffect Dependency Array: Passing [search] causes the effect to run on initial mount AND whenever the user modifies the search input.

• List Keys: Using key={student._id} gives each DOM row a stable identity so React's Virtual DOM reconciliation algorithm only updates changed nodes.

4. Top Technical Interview Questions & Answers

Ø=Ü¡ Q1: What happens step-by-step when a user adds a student?
Answer: 1. User types in fields; handleChange updates formData. 2. Clicking Save fires handleSubmit; e.preventDefault() prevents page reload. 3. addStudent(formData) sends an HTTP POST request to Express. 4. Express validates and saves to MongoDB Atlas. 5. On 201 response, React hides the modal, shows a green alert, and calls fetchStudents() to refresh the table.

Ø=Ü¡ Q2: Why use Axios instead of native fetch()?
Answer: Axios parses JSON automatically (no res.json() needed), automatically rejects promises on 4xx/5xx HTTP error codes for streamlined error handling, and supports timeout configurations out of the box.

Ø=Ü¡ Q3: How does the real-time search work?
Answer: The search input is bound to the "search" state. When changed, useEffect triggers fetchStudents(search). The service appends ?search=term to the GET request. Express runs a case-insensitive MongoDB regex query on fullName and studentId.

Ø=Ü¡ Q4: How does the Edit feature pre-fill student details?
Answer: Clicking Edit calls openEditModal(student). It sets editId to student._id and fills formData with the existing values. Upon submission, handleSubmit checks if editId is present and executes HTTP PUT instead of POST.

5. 60-Second Interview Elevator Pitch

"The frontend is a Single Page Application built with React 18 and bundled with Vite for fast HMR. I structured the codebase using the Service Layer design pattern in services/studentService.js to isolate Axios network calls from the UI components. In App.jsx, I used useState to manage the student roster, controlled forms, and modals, while useEffect coordinates data retrieval and debounced search filtering. The system supports full CRUD operations with live feedback, confirmation guards for deletions, and real-time synchronization with MongoDB Atlas."